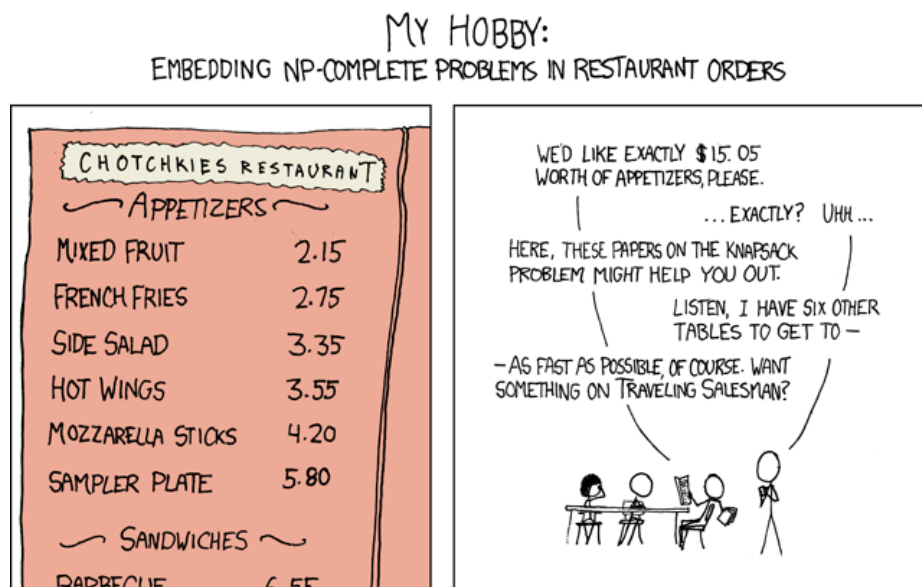


Chapitre 9

Que faire avec des problèmes NP-complets?



1 Algorithmes Probabilistes

Définition - Algo probabiliste

Un algo probabiliste est un algorithme (suite d'instruction finie) dans lequel on peut faire des choix aléatoires. Ainsi, deux exécutions d'un même algo probabiliste sur une même entrée peuvent donner deux résultats différents avec ou non deux temps d'exécution différents.

1.1 Las Vegas

Définition - Las Vegas

Un algorithme est dit de type *Las Vegas* s'il renvoie toujours une valeur correcte mais **son temps d'exécution dépend de l'aléa**.

Remarque : En pratique, on s'intéressera à l'espérance de son temps de calcul comme valeur de complexité.

EXEMPLE 1. Quicksort - « *Random pivoting* »

Définition - Espérance conditionnelle

Soient (Ω, T, P) un espace probabilisé, $A \in T$ un évènement, X une v.a.d sur (Ω, T, P) telle que $X \in L^1$.
On définit l'espérance conditionnelle de X sachant A , notée $E(X | A)$, par

$$E(X | A) = \sum_{x \in X(\Omega)} xP(X = x | A)$$

Théorème de l'espérance totale

Soient (Ω, T, P) un espace probabilisé, $(A_n)_{n \in \mathbb{N}}$ un système quasi-complet d'évènements.
Alors,

$$X \in L^1 \Leftrightarrow (xP(X = x | A_n)P(A_n))_{(x,n) \in X(\Omega) \times \mathbb{N}} \text{ est sommable}$$

et dans ce cas :

$$E(X) = \sum_{n \in \mathbb{N}} E(X | A_n)P(A_n)$$

Théorème

Si X_n représente le nombre de comparaisons effectuées par le tri rapide randomisé sur un tableau de taille n alors
 $E(X_n) \sim 2n \ln(n)$

Remarque : Ici, on ne sortira jamais les arguments de sommabilité vu que **les sommes sont finies!** DÉMONSTRATION.

On pose Y la v.a qui correspond à la position du pivot après la partition du tableau par rapport à celui-ci. Ainsi

$$Y(\Omega) = [0, n-1]$$

$$\forall k \in Y(\Omega), P(Y = k) = \frac{1}{n} \text{ (tirage uniforme)}$$

D'après la formule des espérances totales, (cf. dessin1)

$$\begin{aligned} E(X_n) &= \sum_{k=0}^{n-1} \underbrace{P(Y = k)}_{=\frac{1}{n}} \underbrace{E(X_n | Y = k)}_{n + E(X_k) + E(X_{n-k-1})} \\ &= \sum_{k=0}^{n-1} \frac{1}{n} (n + E(X_k) + E(X_{n-k-1})) \end{aligned}$$

On obtient (en notant pour tout n , $u_n = E(X_n)$) :

$$u_n = n + \frac{2}{n} \sum_{k=0}^{n-1} u_k$$

On considère alors :

$$\begin{aligned}(n+1)u_{n+1} - nu_n &= (n+1)^2 + 2 \sum_{k=0}^n u_k - n^2 - 2 \sum_{k=0}^{n-1} u_k \\ &= 2n+1 + 2u_n\end{aligned}$$

Ainsi,

$$\begin{aligned}(n+1)u_{n+1} - (n+2)u_n &= 2n+1 \\ \Rightarrow \frac{u_{n+1}}{n+2} - \frac{u_n}{n+1} &= \frac{2n+1}{(n+1)(n+2)} \sim \frac{2}{n}\end{aligned}$$

On retrouve le terme général de la série harmonique, qui est une série divergente!

Alors, d'après le théorème de sommation des relations de comparaisons (Cas divergent) :

$$\sum_{k=0}^n \frac{u_{k+1}}{k+2} - \frac{u_k}{k+1} \underset{n \rightarrow +\infty}{\sim} 2H_n$$

i.e (téléscopage et $u_0 = 0$)

$$\frac{u_{n+1}}{n+2} \underset{n \rightarrow +\infty}{\sim} 2\ln(n)$$

D'où $u_n = E(X_n) \sim 2n\ln(n)$

EXEMPLE 2. Trouver le k^e élément

Sélection du k^e elt dans un tableau

- **Entrée :** tableau de taille n , $k \in \llbracket 0, n-1 \rrbracket$
- **Sortie :** L'élément du tableau x qui serait en k^e position si le tableau était trié par valeurs croissantes.

Notre algorithme respecte alors la procédure suivante :

1. On tire uniformément un pivot dans le tableau.
2. On partitionne le tableau autour du pivot (comme dans le Quicksort) qui se retrouve alors en position i .
3. De là, trois possibilités s'offrent à nous :
 - 1^{er} cas, $k = i$: on renvoie alors le pivot!
 - 2^{ème} cas, $k < i$: on cherche le k^e élément dans le tableau des éléments plus petits que notre pivot.
 - 3^{ème} cas, $k > i$: on cherche le $k - i - 1^e$ éléments dans le tableau à droite de i .

Si on note X_n la variable aléatoire qui compte le nombre d'étapes pour sélectionner dans un tableau de taille n . On peut montrer par rec. sur $n \geq 1$ que $E(X_n) \leq 4n$. Si on pose Y la position du pivot après la partition.

$$\begin{aligned}E(X_n) &= \sum_{i=0}^{n-1} P(Y = i) E(X_n | Y = i) \\ &= \sum_{i=0}^{n-1} \frac{1}{n} (n + \max(E(X_i), E(X_{n-1-i}))) \\ &\leq \sum_{i=0}^{n-1} \frac{1}{n} (n + 4 \max(i, n-1-i)) \\ &\dots \\ &\leq 4n\end{aligned}$$

1.2 Monte Carlo

1.2.1 Définition et quelques exemples simples

Définition

Un algorithme probabiliste est de type *Monte Carlo* lorsque son temps d'exécution est garanti mais le résultat peut être faux avec une certaine probabilité.

EXEMPLE 3. Un problème de matrices

On considère le problème suivant :

Problème de matrices

- **Entrée :** $A, B, C \in \mathcal{M}_n(\mathbb{Q})^a$
- **Sortie :** Renvoyer **true** ssi $AB = C$

a. Il paraît difficile de représenter des réels en machine...

On choisit alors $X \in \{0; 1\}^n$ au hasard.

On calcule $A(BX)$ et CX (en $\mathcal{O}(n^2)$) et on renvoie **true** ssi on a obtenu le même résultat.

- Si l'algo renvoie **false**, alors on est sûr que $AB \neq C$
- La probabilité d'erreur est $P_{X \in \{0; 1\}^n}(ABX = CX)$ alors que $AB \neq C$

On se pose donc la question suivante : Soit $M \neq 0(AB - C)$, $P_{X \leftarrow_u \{0, 1\}^n}(MX = 0_n)$?

$\exists i, k$ tq. $M_{i,k} \neq 0$, soit $\sum_{r=1}^n M_{i,r} X_r = M_{i,k} X_k + Z$ (avec Z le reste de la somme).

Il vient alors :

$$\begin{aligned} P((MX)_i = 0) &= P((MX)_i = 0 | Z = 0)P(Z = 0) + P((MX)_i = 0 | Z \neq 0)P(Z \neq 0) \\ &= \frac{1}{2}P(Z = 0) + P(M_{i,k}X_k + Z = 0 | Z \neq 0) \times P(Z \neq 0) \\ &\leq \frac{1}{2}P(Z = 0) + P(X_k = 1 | Z \neq 0)P(Z \neq 0) \text{ car } X_k \perp\!\!\!\perp Z \\ &\leq \frac{1}{2} \end{aligned}$$

d'où

$$P(MX = 0_N) \leq P((MX)_i = 0) \leq \frac{1}{2}$$

BILAN :

- Si l'algo renvoie **false** alors on est sûr que $AB \neq C$.
- Si l'algo renvoie **vrai** alors proba d'erreur $\leq \frac{1}{2}$

Définition

Si l'algo renvoie faux alors qu'il devait renvoyer vrai, on parle de **faux négatif**. On parle de même de **faux positif**.

Théorème

Si on dispose d'un algo type MC sans faux négatif (resp. sans faux positif) avec proba d'erreur $p < 1$ et de complexité C alors on peut obtenir un algo de MC avec complexité nc et erreur de p^n pour tout $n \in \mathbb{N}$

DÉMONSTRATION. On va considérer l'algo suivant,

```
Require:  $x$ 
for  $i \leftarrow 0$  to  $n$  do
  lancer l'algo de MC sur  $x$ 
  if l'algo renvoie false then
    return false
return false
```

Ainsi, la procédure renvoie **false** si au moins une exécution a renvoyé **false** et c'est la bonne valeur car pas de faux négatif. Si la procédure renvoie une valeur erronée alors c'est que l'algo de *MC* s'est trompé de manière indépendante et ceci se produit avec une proba = p^n .

Proposition

Soit $p \in]0; 1[$ et soit A un algo de *MC* avec une complexité f et une erreur p .
On suppose de plus qu'on dispose d'une fonction de vérification v de complexité g qui permet de décider si le résultat de A sur x est correct.
Dans ce cas, il existe un algorithme de *LV* qui résout le pb (sans erreur) avec une espérance de temps de calcul $= \frac{1}{1-p}(f + g)$.

DÉMONSTRATION. On considère la procédure suivante,

```
while true do
   $y \leftarrow A(x)$ 
  if  $v(x, y)$  then
    return  $y$ 
```

Soit X la v.a qui représente le nombre de tours de la boucle **while**, alors $X \hookrightarrow \mathcal{G}(1 - p)$ donc $E(X) = \frac{1}{1-p}$ et chaque tour de boucle a un coût de $(f + g)$.

EXEMPLE 4. On considère un tableau contenant des 0 et des 1 avec autant de 1 que de 0 et on cherche i tq. $T[i] = 1$
→ L'algo qui tire un indice uniformément fournit un algo de *MC* avec complexité 1 et $p = \frac{1}{2}$.
→ On peut vérifier le résultat avec complexité 1.

Bilan : Il existe un algo de *LV* dont l'espérance du temps de calcul vaut $\frac{1}{1-\frac{1}{2}}(1 + 1) = 4$. Cet algorithme consiste à tirer au hasard un indice jusqu'à obtenir $T[i] = 1$.

EXEMPLE 5. Élément majoritaire. Soit t un tableau de taille n qui contient au moins un élément qui apparaît strictement plus de $\lfloor \frac{n}{2} \rfloor$. On veut trouver cet elt.
Si on considère l'algorithme de *MC* consistant à tirer un indice au hasard alors on a une proba d'erreur $p < \frac{1}{2}$.
On peut vérifier si un elt est majoritaire en $\mathcal{O}(n)$ et on obtient l'algo de *LV* consistant à tirer au hasard et vérifier en $\mathcal{O}(n)$.

1.2.2 Exemple d'algorithme de *MC* avancé

Perfect Matching

Soit $G = (S = S_1 \cup S_2, A)$ un graphe biparti ($n = |S_1| = |S_2|$).
On cherche à savoir s'il existe un couplage de taille n .
On définit la matrice $M = (m_{i,j})$ avec $m_{i,j} = x_{i,j} \delta_{(i,j) \in A}$ où $x_{i,j}$ est une variable :

1. $f : M \mapsto \det(M) \equiv \tilde{0} \Leftrightarrow$ il n'existe pas de couplage parfait
2. Pour décider si $f \equiv \tilde{0}$, on se contentera de l'évaluer en choisissant n^2 valeurs au hasard.

DÉMONSTRATION.

1. — D'une part, $\exists CP \Leftrightarrow \exists \sigma \in S_n$ tq. $\forall i \in \llbracket 1; n \rrbracket, m_{i,\sigma(i)} = x_{i,\sigma(i)} \neq 0$. Ainsi, s'il n'existe pas de *CP* alors tous les termes du déterminant sont nuls :

$$\det(M) = \sigma_{\sigma \in S_n} \epsilon(\sigma) \prod_{i=1}^n m_{i,\sigma(i)} = 0$$

- Réciproquement, s'il existe un *CP*, on peut fixer σ tq. $\forall i, m_{i,\sigma(i)} = x_{i,\sigma(i)} \neq 0$ et $\prod_{i=1}^n x_{i,\sigma(i)} \neq 0$ et sera alors un terme du déterminant obtenu qui ne sera pas la fonction nulle.

$\det(M(x_{1,1}, \dots, x_{n,n}))$ est fonction polynomiale de plusieurs variables.

2. Soit P un polynôme non-nul à n variables de degré d et soit $|\Omega| = m$ alors

$$P_{R(a_1, \dots, a_n) \in \Omega^n} (P(a) = 0) \leq \frac{d}{m}. \quad (1)$$

où $\Omega \subset F$ avec F un corps fini dans lequel les coeffs. du polynômes sont définis.

→ revoir la notion de degré d'un polynôme multivarié.

Dans notre cas particulier $d = n$. Si on fixe un ensemble fini de taille $2n$ et qu'on évalue au hasard notre déterminant sur cet ensemble alors on obtient une erreur de proba $\frac{1}{2}$.

Bilan :

Soit $G = (S = S_1 \cup S_2, A)$. On va construire M_{al} ainsi : $\forall (i, j) \in \llbracket 1; n \rrbracket^2$, si $(i, j) \in A$, on tire $r \in \{1, \dots, 2n\}$ et on pose $M_{al(i,j)} = r$, sinon $M_{al(i,j)} = 0$.

Si $\det(M_{al}) = 0$ on renvoie qu'il n'y a pas de couplage parfait et sinon on renvoie **true**.

Analyse : pas de faux positif

$$P_R(\text{faux alors qu'il existe CP}) \leq \frac{1}{2}$$

Lemme de Schwarz-Zippel : rec. sur n le nombre de variable

1.3 Échantillonnage

1.3.1 Comment mélanger un tableau?

Mélange de Knuth

for each case i **do**

On tire un indice $k \in \llbracket 0; i \rrbracket$ et on échange les cases k et i .

```
1 void shuffle(int* tab, int n){
2     for (int i=0; i<n; i++){
3         int k = rand()%(i+1);
4         swap(tab,i,k);
5     }
6 }
```

Proposition - Correction de l'algorithme de Knuth

On désigne l'invariant suivant. En notant t_{init} le tableau avant modification et t la tableau au i^e tour.

Pour $i \in \llbracket 0; n-1 \rrbracket$, si $\sigma \in \mathcal{S}_{\llbracket 0; i \rrbracket}$ alors

$$P(t(j) = t_{init}(\sigma(j)), \forall j \in \llbracket 0; i \rrbracket) = \frac{1}{(i+1)!}$$

C'est à dire que les $i+1$ premières cases ont été mélangées suivant σ avec proba $\frac{1}{(i+1)!}$.

DÉMONSTRATION.

Montrons ce résultat par récurrence :

- $i = 0$: OK
- $H_n \Rightarrow H_{n+1}$
 - **1er cas** : $\sigma(i+1) = i+1$

On obtiendra σ au $i+1$ tour ssi on a tiré l'indice $i+1$ ce qui arrive avec proba $\frac{1}{i+1}$ et que la 1^e partie est organisée suivant $\sigma_{\llbracket 0; i \rrbracket}$ ce qui est vrai avec proba $\frac{1}{(i+1)!}$ par H.R appliquée à $\sigma_{\llbracket 0; i \rrbracket}$.
Les tirages étant indépendants, on obtient bien $P(t(j) = t_{init}(\sigma(j)), \forall j \in \llbracket 0; i+1 \rrbracket) = \frac{1}{(i+2)!}$
 - **2ème cas** : $\sigma(i+1) = k < i+1$

On pose j tq $\sigma(j) = i+1$. On est dans cette situation ssi on a tiré j pour l'indice $i+1$ ce qui arrive avec une proba $\frac{1}{i+2}$ et au tour précédent on avait obtenu σ' def. sur $\llbracket 0; i \rrbracket$ par $\forall r \neq j, \sigma'(r) = \sigma(r)$ et $\sigma(j) = k$ et la proba de cet évènement vaut $\frac{1}{(i+1)!}$ par H.R. Par indépendance on a bien la probabilité souhaitée comme précédemment.

Ce qui clôt la récurrence, d'où le résultat voulu.

1.3.2 Comment sélectionner k cases d'un tableau ou k elts dans une liste de taille n avec proba $\frac{1}{\binom{n}{k}}$

➤ On remarque qu'il suffit de mélanger uniformément le tableau et de récupérer les k premiers éléments.

En effet, si on considère $A \subset \llbracket 0; n-1 \rrbracket$ de taille k ,

$$\begin{aligned} P(\text{obtenir } A) &= P(\text{obtenir une permutation positionnant tous les éléments de } A \text{ dans } \llbracket 0; k-1 \rrbracket) \\ &= \frac{\text{nombre de permutation où les elts des } k \text{ premières cases sont ceux de } A}{n!} \\ &= \frac{k!(n-k)!}{n!} \\ &= \frac{1}{\binom{n}{k}} \end{aligned}$$

Remarque : si $k = 1$ alors cela revient à sélectionner uniformément un elt.

```
1 let rec select (e : int) (liste : int list) (t : int) : int =
2   match liste with
3   | [] -> e
4   | h::next -> let y = Random.int (t+1) in
5                 if y=0 then select h next (t+1)
6                 else select e next (t+1)
7
8 let selection (l : int list) : int =
9   match l with
10  | [] -> failwith "vide"
11  | a::suite -> select a suite
12
13 let selection_k (k : int) (liste : int list) : int array =
14   let res = Array.make k 0 in
15   let rec aux (l : int list) (i : int) : int list =
16     (* recopie les k premiers elts de liste et renvoie la liste a partir de l'indice k*)
17     if (i < k) then
18       match l with
19       | [] -> failwith "trop courte"
20       | h::t -> res.(i) <- h; aux t (i+1)
21     else l
22   in
23   let res = aux liste 0 in
24
25   let rec aux2 (l : int list) (t : int) : int array =
26     (* Elle parcourt le reste de la liste et d'echange les elt avec ce qui precede n'effectuant que les
27       echanges des cases [|0;k-1|] *)
28     match l with
29     | [] -> res
30     | h::next -> let y = Random.int (t+1) in
31                   if y < k then res.(y) <- h;
32                   aux2 next (t+1)
33   in aux2 res k
```

2 Algo d'approximation

Problème d'optimisation

Définition

Soient E un ensemble d'entrée, S un ensemble de sol, $\mathcal{R} \subset E \times S$ et $c : S \rightarrow \mathbb{R}$.

Minimisation :

Soit $x \in E$.

Renvoyer $y \in S$ tq.

$$\mathcal{R}(x, y) \text{ et } c(y) = \min_{z \in S: \mathcal{R}(x, z)} (c(z))$$

Maximisation :

Soit $x \in E$.

Renvoyer $y \in S$ tq

$$\mathcal{R}(x, y) \text{ et } c(y) = \max_{z \in S: \mathcal{R}(x, z)} (c(z))$$

Pour un problème d'optimisation avec un pb. de seuil associé NP-complet on peut envisager de construire un algo qui fournit une α approximation, i.e

Minimisation :

Renvoyer $y \in S$ tq
 $\mathcal{R}(x, y)$ et $c_{opt}(x) \leq c(y) \leq \alpha c_{opt}(x)$
 avec $\alpha > 1$

Maximisation :

Renvoyer $y \in S$ tq
 $\mathcal{R}(x, y)$ et $\alpha c_{opt}(x) \leq c(y) \leq c_{opt}(x)$
 avec $\alpha < 1$

Définition

Soit A un algo qui pour tout $x \in E$ renvoie $y \in S$ tq. $\mathcal{R}(x, y)$ alors le coefficient d'approximation du pb de minimisation est

$$\max_{x \in E} \frac{c(A(x))}{c_{opt}(x)} = \alpha$$

Définition - Pour la maximisation

Le coeff d'approx est

$$\alpha = \min_{x \in E} \frac{c(A(x))}{c_{opt}(x)}$$

EXEMPLE 6. MinVertexCover

Soit $G = (S, A)$ on veut trouver $S' \subset S$ qui réalise une couverture par sommets de taille minimale.

Proposition 1

Un couplage maximal de G fournit une couverture par sommets composée des sommets adjacents au couplage.

Proposition 2

Si on dispose d'un couplage à k arêtes alors toute couverture par sommets est de taille au moins k .

CCL : Si on a un couplage maximal de taille k alors les $2k$ sommets le composant forment une couverture par sommet (prop 1) et la couverture optimale est de taille au moins k (prop 2).

$$2c_{opt} \geq 2k = c_{algo} \geq c_{opt} \geq k$$

Ainsi trouver un couplage maximal fournit une 2-approximation au problème MVC ce qui peut être fait aussi avec

$A' \leftarrow \emptyset$

$S' \leftarrow \emptyset$

while A contient au moins une arête (x, y) avec $x \notin S', y \in S$ **do**

 sélectionner une telle arête (x, y)

$A' \leftarrow A' \cup \{(x, y)\}$

$S' \leftarrow S' \cup \{(x, y)\}$

return S'

2.1 minVC

$G = (S, A)$ non-orienté. Chercher $V \subset S$ de taille minimale tq. $\forall a \in A, a \cap V \neq \emptyset$. Un couplage maximal fournit une couverture par sommets de taille $\leq 2 \times$ taille optimale.

1) couplage max $\Rightarrow$ couv. par sommets. Soit C un couplage maximal. Alors

$$V = \bigcup_{(x,y) \in C} \{x, y\}$$

alors si V n'est pas une couverture par sommet c.a.d qu'il existe $a = (x', a') \in A$ tq. $x' \notin V$ et $y' \notin V$ et dans ce cas $C \cup \{a\}$ serait un couplage ce qui contredit la maximalité de C .

2) Si on dispose d'un couplage avec k arêtes alors toute couverture par sommet est de taille au moins k . Si on considère V une couverture par sommet, elle doit couvrir chaque arête de C avec un sommet $\neq$ à chaque fois et donc elle a au moins k sommets.

Conclusion L'algo renvoie $2k$ sommets avec k la taille d'un couplage $\leq$ taille d'une couverture optimale. Ainsi, $2k \leq 2 \times$ taille d'une couv. optimale.

2.2 Somme Partielle

On définit le problème

Somme partielle

Entrée : $x_1, \dots, x_n$ des entiers naturels, $C \in \mathbb{N}$

Sortie : $I \subset \llbracket 1; n \rrbracket$ tq $\sum_{i \in I} x_i \leq C$ qui maximise $\sum_{i \in I} x_i \leq C$

Algorithme

On trie les x_i par valeurs décroissantes. Donc $x_1 \geq \dots \geq x_n$.

$S \leftarrow 0$

$I \leftarrow \emptyset$

for $i \leftarrow 0$ **to** n **do**

if $x_i \leq C - S$ **then**

$S \leftarrow S + x_i$

$I \leftarrow I \cup x_i$

return I

Cet algo fournit une $\frac{1}{2}$ approximation au problème, i.e

$$\text{somme renvoyée} \geq \frac{\text{somme optimale}}{2}$$

On va en fait montrer que somme renvoyée $\geq \frac{C}{2}$.

Soient $x_1, \dots, x_n$ et C

1er cas : $\sum_{i=1}^n x_i \leq C$ et dans ce cas l'algo renvoie la solution optimale.

2ème cas : Si $\sum_{i=1}^n x_i > C$. On considère $j \in \llbracket 1; n \rrbracket$ le 1er indice tel que $j \notin I_{\text{renvoyé}}$ et $x_j \leq C$. On va montrer que la valeur de $\tilde{S}$ au j^e tour de boucle c.a.d celui tel que $x_j > C - \tilde{S}$ est $\geq \frac{C}{2}$. Ce qui permettrait de conclure car $S_{\text{renvoyée}} \geq \tilde{S} \geq \frac{C}{2}$.

Montrons que $\tilde{S} \geq \frac{C}{2}$. Supposons par l'absurde $\tilde{S} < \frac{C}{2}$, impliquant $x_j > C - \tilde{S} > \frac{C}{2} \Rightarrow x_j > \tilde{S}$. Or comme les $(x_i)_i$ sont considérés par valeurs décroissantes, $x_j \leq \tilde{S}$ car $\tilde{S}$ est une somme non vide composée de nombres ≥ 0 tous $\geq x_j$. **ABSURDE.**

2.3 MaxCut

On considère le problème suivant :

MaxCut

Soit $G = (S, A)$ non-orienté. On cherche $V \subset S$ qui maximise $|A \cap (V \times (S \setminus V))|$.

C'est-à-dire le nombre d'arête qui sont de la forme : un sommet de V et son complémentaire.

EXEMPLE 7. $V = \{2; 3\}$ et $S \setminus V = \{1; 4; 5\}$ fournit une coupure de taille 5.

Algorithme

$V \leftarrow S$

$T \leftarrow \emptyset$

while déplacer un sommet de V vers T ou T vers V augmente strictement la taille de la coupe **do**
 Le déplacer

return V

On va montrer que l'ensemble V renvoyé par l'algorithme a une coupe de taille $\geq \frac{|A|}{2}$ et donc $\geq \frac{|opt|}{2}$.

On va donc montrer que si on a V et T qui forment une partition de S telle que pour tout $x \in S$, si on le change d'ensemble, la taille de la coupe obtenue $\leq$ la taille de la coupe qu'on avait, alors la taille de la coupe qu'on avait $\geq \frac{|A|}{2}$.

Soit $x \in S$, on pose $deg_{cut}(x)$ = nbre d'arêtes de la coupe incidentes à x et $deg_{\neg cut}(x)$ = nbre d'arêtes incidentes à x qui n'est pas dans la coupe. On a $deg(x) = deg_{cut}(x) + deg_{\neg cut}(x)$.

La propriété sur les sommets (l'équilibre local) garantit que

$$\forall x \in S, deg_{cut}(x) \geq deg_{\neg cut}(x)$$

Car déplacer x d'un ensemble vers l'autre crée une variation de la taille de cut de $deg_{\neg cut}(x) - deg_{cut}(x)$. Ainsi,

$$\begin{aligned} \sum_{x \in S} deg(x) &= 2 \times |A| \\ \text{et} \\ \sum_{x \in S} deg_{cut}(x) &\geq \sum_{x \in S} deg_{\neg cut}(x) \\ \Rightarrow \begin{cases} |A_{cut}| \geq |A_{\neg cut}| \\ |A_{cut}| \geq |A_{\neg cut}| = |A| \end{cases} \end{aligned}$$

$$\Rightarrow |A_{cut}| \geq \frac{|A|}{2}$$

2.4 Couplage de poids maximal

On considère le problème suivant :

Couplage de poids maximal

Pour $G = (S, A, p)$ non-orienté pondéré à poids positifs on cherche un couplage C qui maximise $p(C) = \sum_{a \in C} p(a)$

Algorithme

```

 $C \leftarrow \emptyset$ 
for each  $a \in A$  par valeur de poids décroissante do
  if  $C \cup \{a\}$  est un couplage then
     $C \leftarrow C \cup \{a\}$ 
return  $C$ 

```

Cet algo renvoie un couplage C tq. $p(C) \geq \frac{1}{2} p(C^*)$ où C^* est un couplage de poids optimal. Fixons C et C^* . Soit $a \in C^*$, si $a \in C \rightarrow \text{ok}$.

Si $a \notin C$, alors il existe $a' \in C$ tq. $p(a') \geq p(a)$ et $a' \cap a \neq \emptyset$. Ainsi $\forall a \in C^*, \exists a' \in C$ (on peut avoir $a' = a$) tq. $a' \cap a \neq \emptyset$ et $p(a') \geq p(a)$.

On pose

$$g: \begin{cases} C^* & \rightarrow C \\ a & \mapsto a' \end{cases}$$

Pour chaque $a' \in C$, elle admet au plus 2 antécédents par g : une pour chacune de ses extrémités

$$p(C^*) = \sum_{a \in C^*} p(a) \leq \sum_{a \in C^*} p(g(a)) \leq 2 \sum_{a \in C^*} p(a) = 2 \times p(C)$$

car p est à valeurs dans $\mathbb{R}_+$

2.5 Problème de voyageur de commerce

On considère le problème suivant :

Voyageur de commerce

Entrée : un graphe complet de taille n pondéré

Sortie : Un cycle hamiltonien de poids minimal

Théorème

Si la fonction de pondération satisfait l'inégalité triangulaire alors il existe un algo polynomial qui fournit une 2-approximation au problème suivant.

Remarques :

- Si C est un cycle hamiltonien et qu'on lui retire une arête alors on obtient un arbre couvrant.
- Si p satisfait l'inégalité triangulaire, alors elle est à valeurs positives.
- Soient T un cycle hamiltonien de poids minimal et A un arbre couvrant de poids minimal : $p(A) \leq p(T)$

Ainsi, à partir d'un arbre couvrant de poids minimal A , on va chercher à construire un cycle hamiltonien C de poids $\leq 2p(A) \leq 2p(T)$.

CF DESSIN

Si on considère un parcours en profondeur de l'arbre (en rouge), on obtient un cycle C' de poids $2p(A)$ passant au moins une fois par chaque sommet :

$$C'' = 1, 2, 3, 5, 9, 10, 11, 6, 7, 3, 16, 12, 13, 8, 14, 15$$

$p(C'') \leq p(C)$ par inégalité triangulaire. Ainsi $p(C'') \leq 2p(A) \leq 2p(T)$. C' est bien un cycle hamiltonien fournissant une 2-approximation.

Difficulté du problème du voyageur de commerce Problème de décision à seuil associé :

PVC_{seuil}

Entrée : graphe complet pondéré, $s \in \mathbb{R}$

Sortie : renvoie **true** ssi il existe un cycle hamiltonien de poids $\leq s$ dans le graphe

hampath

Entrée : Graphe orienté $G = (S, A)$ et $(x, y) \in S^2$

Sortie : renvoie **true** ssi il existe un chemin hamiltonien dans G de x vers y

uhampath

Entrée : graphe non-orienté et $G = (S, A)$ et $(x, y) \in S^2$

Sortie : renvoie **true** ssi il existe un chemin hamiltonien entre x et y .

hamcycle

Entrée : graphe non-orienté

Sortie : **true** ssi il existe un cycle hamiltonien

$\Rightarrow$ On va montrer que

$$3SAT \leq_p \text{hampath} \quad (2)$$

$$\text{hampath} \leq_p \text{uhampath} \quad (3)$$

$$\text{uhampath} \leq_p \text{hamcycle} \quad (4)$$

$$\text{hamcycle} \leq_p \text{PVC}_{\text{seuil}} \quad (5)$$

5) **hamcycle** $\leq_p$ **PVC_{seuil}** On expose

$$G = (S, A) \rightarrow G' \text{ complet sur } S, p, s = 0$$

$$\forall (x, y) \in A, p(x, y) = 0$$

$$\forall (x, y) \notin A, p(x, y) = 1$$

Si G admet un cycle hamiltonien alors G' admet un cycle hamiltonien ne passant que par des arêtes de poids 0 et donc le poids du cycle est nul.

Si G' admet un cycle hamiltonien de poids ≤ 0 . Les poids étant positifs, la seule possibilité est que toutes les arêtes du cycle soient de poids nuls et donc dans A , i.e le cycle est aussi un cycle hamiltonien pour G .

4) **uhampath** $\leq_p$ **hamcycle** On expose

$$G = (S, A), (x, y) \in S^2 \rightarrow G' = (S \cup \{new\}, A')$$

$$\text{avec } new \notin S, A' = A \cup \{(x, new), (y, new)\}$$

Si G admet un chemin hamiltonien :

$$x = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_n = y$$

Alors :

$$x = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_n = y \rightarrow new \rightarrow x$$

est un cycle hamiltonien de G' .

Réciproquement, si G' admet un cycle hamiltonien qui passe nécessairement par new qui est connecté uniquement à x et y donc le cycle est de la forme (à sens de parcours près) :

$$new \rightarrow x \rightarrow \dots \rightarrow y$$

et donc on obtien bien un chemin entre x et y qui passe exactement une fois par chaque sommet de G et n'emprunte que des arêtes de G .

3) **hampath** $\leq_p$ **uhampath** On expose

$$G = (S, A) \text{ orienté}, (x, y) \in S^2 \rightarrow G' = (S', A')$$

$\forall s \in S$, on crée 3 sommets s^-, s, s^+ tq. $s^- \rightarrow s \rightarrow s^+$ (en termes d'arêtes) et donc pour tout $(x, y) \in A$ on relie x^+ à y^- .

Si G admet un chemin hamiltonien $x = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_n = y$ alors

$$x^- \rightarrow x \rightarrow x^+ \rightarrow x_1^- \rightarrow \dots \rightarrow x_n^- \rightarrow x_n \rightarrow x_n^+$$

est bien un chemin hamiltonien de x^- à y^+ dans G' .

Réciproquement, si on a un chemin hamiltonien G'

$$x^- \rightarrow \alpha \rightarrow \dots \rightarrow y^+$$

avec $\alpha = x$ ou un t^+ tq. $(t, x) \in A$.

Or si on ne commence pas par passer par x alors le chemin devra y passer plus tard en passant nécessairement par x^+ et on serait bloqué en x .

On en déduit que le chemin est de la forme

$$x^- \rightarrow x \rightarrow x^+ \rightarrow x_1^- \rightarrow t^- \rightarrow t \rightarrow t^+$$

avec $(x, t) \in A$. Et de proche en proche (rec. lourde à démontrer), pour les mêmes raisons que précédemment on obtient un chemin de la forme :

$$x^- \rightarrow x \rightarrow x^+ \rightarrow x_1^- \rightarrow \dots \rightarrow x_n^- = x_n^- \rightarrow x_n = y \rightarrow x_n^+ = y^+$$

Ainsi, par construction

$$x \rightarrow x_1 \rightarrow \dots \rightarrow x_n = y$$

est bien un chemin hamiltonien de G .

2) **3SAT** $\leq_p$ **hampath** cf. cahier de prépa + schémas

$\Rightarrow$ Revenons à notre problème

Théorème

Soit $\varepsilon > 0$, si on dispose d'un algo polynomial qui résout le PVC avec erreur $(1 + \varepsilon)$ alors $P = NP$

DÉMONSTRATION. Si un tel algo existe alors on pourrait trouver un algo polynomial pour cycle hamiltonien qui est NP -complet et on aurait $P = NP$.

$$G = (S, A) \rightarrow G' \text{ complet}, p(x, y) = 1 \text{ si } (x, y) \in A \text{ et } n(1 + \varepsilon) + 1 \text{ sinon}$$

On utilise l'algo approché sur G' s'il renvoie une valeur $\leq (1 + \varepsilon)n$ alors G admet un cycle hamiltonien et sinon il n'en a pas.

2.6 MaxSAT

MaxSAT

Étant donnée une formule sous forme CNF, trouver une valuation qui maximise le nb de clauses satisfaites simultanément.

Proposition

Max2SAT est *NP*-complet

⇒ On va donner un algo d'approximation pour MaxSAT obtenu par dérandomisation.

Algorithme de Monte Carlo

On tire une valuation au hasard

Théorème

Soit φ une formule sous forme CNF avec c clauses. Soit $E(\varphi)$ l'espérance du nombre de clauses satisfaites. Alors
Prendre une valuation au hasard $\Rightarrow E(\varphi) \geq \frac{c}{2}$

DÉMONSTRATION.

Pour chaque clause C_i on pose X_i la variable aléatoire qui vaut 1 si C_i est satisfaite (0 sinon). Ainsi, $X_i \hookrightarrow B(p_i)$ avec $p_i \geq \frac{1}{2}$ car il y a au moins un littéral dans la clause qui à lui seul a une proba $\frac{1}{2}$ d'être vraie. Il vient :

$$E(\varphi) = \sum_{i=1}^c E(X_i) = \sum_{i=1}^c p_i \geq \frac{c}{2}$$

Dérandomisation : Soit φ dont les variables sont $x_1, \dots, x_n$. On va choisir pour $i = 1$ à n la valeur de x_i de la manière suivante :

- pour x_1 : On calcule $E(\varphi \mid x_1 = \top)$ et $E(\varphi \mid x_1 = \perp)$ et on choisit la valeur qui maximise cette espérance.
- pour x_i avec $x_1, \dots, x_{i-1}$ déjà valué on calcule $\varphi' = \varphi$ où l'on remplace $x_1, \dots, x_{i-1}$ par leur valuations. On calcule ensuite $E(\varphi' \mid x_i = \top)$ et $E(\varphi' \mid x_i = \perp)$ et on choisit la valeur qui maximise cette espérance.

On remarque qu'étant donnée une valuation partielle v_{i-1} on construit v_i^1 et v_i^2 , on calcule

$$\max(E(\varphi \mid v_i^1), E(\varphi \mid v_i^2))$$

pour déterminer la valuation de x_i .

Si $\varphi = C_1 \wedge \dots \wedge C_k$ alors

$$E(\varphi \mid v_i) = \sum_{j=1}^k P(C_j \text{ sat} \mid v_i)$$

Ainsi, pour notre algo il suffit de savoir calculer :

$$P(C_j \text{ sat} \mid v_i)$$

or

$$\begin{aligned} P(C_j \text{ sat} \mid v_i) &= 1 \text{ si l'un des littéraux défini par } v_i \text{ vaut } \top \\ &= 0 \text{ si tous les littéraux définis par } v_i \text{ valent } \perp \\ &= 1 - \frac{1}{2^k} \text{ si on a } k \text{ littéraux non définis (et aucun défini à } \top) \end{aligned}$$

Théorème

Si φ est sous forme CNF alors la valuation calculée par cet algo satisfait au moins $E(\varphi)$ clauses.

DÉMONSTRATION. On montre par rec. sur i que $E(\varphi \mid v_i) \geq E(\varphi)$:

— Si $i \in \mathbb{N}$, notons $H_i = E(\varphi \mid v_i) \geq E(\varphi)$

— **Initialisation**

— $i = 0$: vrai

— $i = 1$:

$$E(\varphi) = \frac{1}{2}E(\varphi \mid x_1 = \top) + \frac{1}{2}E(\varphi \mid x_1 = \perp)$$

donc le max $\geq E(\varphi)$

— $H_i \Rightarrow H_{i+1}$

$$E(\varphi \mid v_i) = \frac{1}{2}E(\varphi \mid v_i \text{ et } x_{i+1} = \top) + \frac{1}{2}E(\varphi \mid v_i \text{ et } x_{i+1} = \perp)$$

On a donc l'une des 2 valeurs qui est bien supérieure ou égale à $E(\varphi \mid v_i)$ et donc à $E(\varphi)$ par H.R.

Bilan L'algo déterministe satisfait lui aussi au moins la moitié des clauses et c'est donc un algorithme d'approximation de facteur $\frac{1}{2}$.

3 Branch and Bound

Bruteforce

1. **Backtracking** $\rightarrow$ pb de recherche/pb de décision

2. **Branch and bound** (séparation et évaluation) $\approx$ adaptation du backtracking à des problèmes d'optimisation.

Principe : On fait une recherche exhaustive d'une solution où on construit la solution de telle sorte qu'on coupe la branche (on arrête le programme) dès qu'une solution partielle n'a plus aucune chance de donner lieu à une solution correcte en la prolongeant.

MaxSat $\varphi \rightarrow$ valuation qui maximise le nombre de clauses satisfaites.

Pour un pb de maximisation

Entrée : x

Sortie : y tq xRy et $f(x, y)$ maximale

Ici $\varphi Rv \Leftrightarrow v$ est une valuation des var. de φ et $f(v, \varphi) =$ le nombre de clauses de φ satisfaites avec v .

Algorithme (Version sans élagage)

```

max  $\leftarrow -\infty$ 
 $y \leftarrow \emptyset$ 
def aux( $\tilde{y}$ ) :
  if  $\tilde{y}$  est totale then
    if  $f(\tilde{y}, x) > \text{max}$  then
      max  $\leftarrow f(\tilde{y}, x)$ 
       $y \leftarrow \tilde{y}$ 
  else
    for chaque prolongement de  $\tilde{y}$  en  $y'$  do
      aux( $y'$ )
end def aux
aux( $y$ )
return  $y$ 

```

On va définir une heuristique h qui respecte la propriété suivante :

Proposition

Étant donné $\tilde{y}$ une solution partielle du pb sur l'entrée x alors $\forall y'$ sol. qui prolonge $\tilde{y}$:

$$f(y', x) \leq h(\tilde{y})$$

Pour **MaxSat**, on peut prendre

$$h(\tilde{v}) = \text{nb total de clauses dont tous les littéraux sont faux pour } \tilde{v}$$

On expose le pseudo code de la version avec élagage :

Algorithme (Version avec élagage)

```
max ← -∞
y ← ∅
def aux( $\tilde{y}$ ) :
  if  $\tilde{y}$  est totale then
    if  $f(\tilde{y}, x) > \text{max}$  then
      max ←  $f(\tilde{y}, x)$ 
      y ←  $\tilde{y}$ 
    else if  $h(\tilde{y}) > \text{max}$  then
      for chaque prolongement  $\tilde{y}$  en  $y'$  do
        aux( $y'$ )
  end def aux
aux(y)
return y
```

Si on travaille sur un pb de minimisation :

On va définir une heuristique h qui respecte la propriété suivante :

Proposition

Étant donné $\tilde{y}$ une solution partielle du pb sur l'entrée x alors $\forall y'$ sol. qui prolonge $\tilde{y}$:

$$f(y', x) \geq h(\tilde{y})$$